

MARKOS AI — Master Build Specification (Detailed, One-Go Build Document)

Audience: an autonomous coding agent (Codex / Claude) working in VS Code. **Purpose:** the single, complete source of truth to build **MARKOS AI v1.0** end-to-end for the **Bahrain market**. Read this entire document before writing any code. Where this conflicts with a prior assumption, **this document wins**. It supersedes the PRD where they differ and is the engineering companion to the Implementation Plan and the Figma Design Document. This is written so that a single developer directing you can build the whole product from this one file. It contains the exact stack, repo layout, environment, database schema, API catalogue, the eight AI agents with their prompt and output contracts, the Instagram publishing state machine, the Bahrain payment + VAT model, security, the milestone build order, and the acceptance criteria for every step.

0. Agent Operating Instructions (READ FIRST)

You are building **MARKOS AI** — an AI marketing operating system that replaces the marketing function for small businesses in **Bahrain**. v1.0 is **Instagram-only**. It learns a business during onboarding, stores that knowledge permanently in a **Knowledge Vault** (RAG over pgvector), and uses **eight AI agents** to generate strategy and content, schedule and publish to Instagram, analyze performance, and advise — all bilingual (**Arabic + English, RTL-aware**).

0.1 How to execute this build

1. **Build in milestones, in order (Section 22): M0 → M6.** Do not start a milestone until the previous one's acceptance criteria pass. Within a milestone, work in **vertical slices**: DB → API → AI (if needed) → web UI → tests.
2. **Build the foundation (Section 7) before any feature**, and within features build the **Knowledge Vault + RAG (Section 12) before the agents (Section 13)** — every agent depends on it.
3. **After each slice, run `make verify`** (typecheck + lint + tests for all services). Keep it green. Commit with Conventional Commits (Section 6.7). Never move on with red CI.
4. **Self-check against acceptance criteria.** Each milestone and most sections end with explicit "Done when..." criteria. Treat them as the gate. If you cannot meet a criterion, stop and surface the blocker rather than faking it.

5. **When this document is silent**, choose the smallest, most conventional option consistent with the stack in Section 3, and record the decision in a code comment and in `/docs/decisions.md`.

0.2 Non-negotiable invariants (violating any of these is a build failure)

- `workspace_id` isolation is sacred. Every workspace-owned query is scoped to the caller's workspace; add an isolation test per table (Section 9). No cross-workspace read or write, ever.
- TypeScript `strict` everywhere; typed Python (`mypy`) in the AI service. No `any`.
- **Models are configuration, not constants.** Every LLM/embedding/image call goes through the provider interface (Section 13.1) with model IDs from env. Never hardcode a model name inside an agent.
- **Money is integer minor units + ISO-4217 currency.** Never floats. Default currency **BHD** (3 decimal places → store fils, i.e. amount × 1000).
- **Respect Instagram's real limits** (Section 18). Publishing is a defensive state machine, not a fire-and-forget call.
- **Meter every AI token and image** from day one; enforce plan quotas (Section 19).
- **No secrets, URLs, or user-facing strings hardcoded.** Secrets from env / AWS Secrets Manager; copy from `packages/i18n`.
- **Bilingual + RTL from the first screen.** Every screen works in Arabic (RTL) and English.

0.3 Global Definition of Done

A feature is done when: `make verify` is green; it works end-to-end in the web app in **both languages**; workspace isolation holds; AI usage is metered and quota-enforced; all states (empty / loading / error / success / limit-reached) are implemented; and it has unit + integration tests (including a tenant-isolation test for any workspace-owned table, and a critical-path test for any money or publishing path).

1. What We Are Building

1.1 The closed loop (the product's whole point)

```
Onboarding → Knowledge Vault (permanent business memory)
  → AI Strategy (30/60/90 + pillars) → AI Content Plan (monthly calendar)
  → AI Content Creation (captions, hashtags, carousels, reels, images)
  → Schedule → Publish to Instagram → Analytics sync
  → AI interprets performance → feeds back into the Vault → (loop tightens)
```

Every milestone moves this loop closer to closing. Ship the loop, not isolated features.

1.2 The eight AI agents

- 1. **Marketing Strategist** — 30/60/90-day strategy + content pillars.
- 2. **Content Planner** — monthly calendar, content-type mix, best-time-to-post.
- 3. **Content Creator** — captions, hashtags (niche/mid/broad), CTAs, carousel copy.
- 4. **Reel Script** — hook → body → CTA scripts + concept briefs.
- 5. **Image Prompt** — brand-aligned prompts for the image model.
- 6. **Analytics Consultant** — plain-English interpretation of Instagram metrics.
- 7. **Recommendation Engine** — proactive content/timing/campaign suggestions.
- 8. **Business Growth Advisor** — strategic advice grounded in the full Vault.

1.3 Plans (Bahrain pricing, VAT-exclusive)

Plan	USD	BHD (+VAT)	Workspaces	AI gens/mo	AI images/mo	Strategies/mo	Posts/mo	Team seats
Starter	\$49	BD 18	1	100	20	1	30	1
Growth	\$99	BD 37	1	300	60	3	90	2
Premium	\$199	BD 75	3	700	150	12 (fair-use)	200	5
Enterprise	\$499	BD 188	10	1,500 (fair-use)	500	20 (fair-use)	500	15

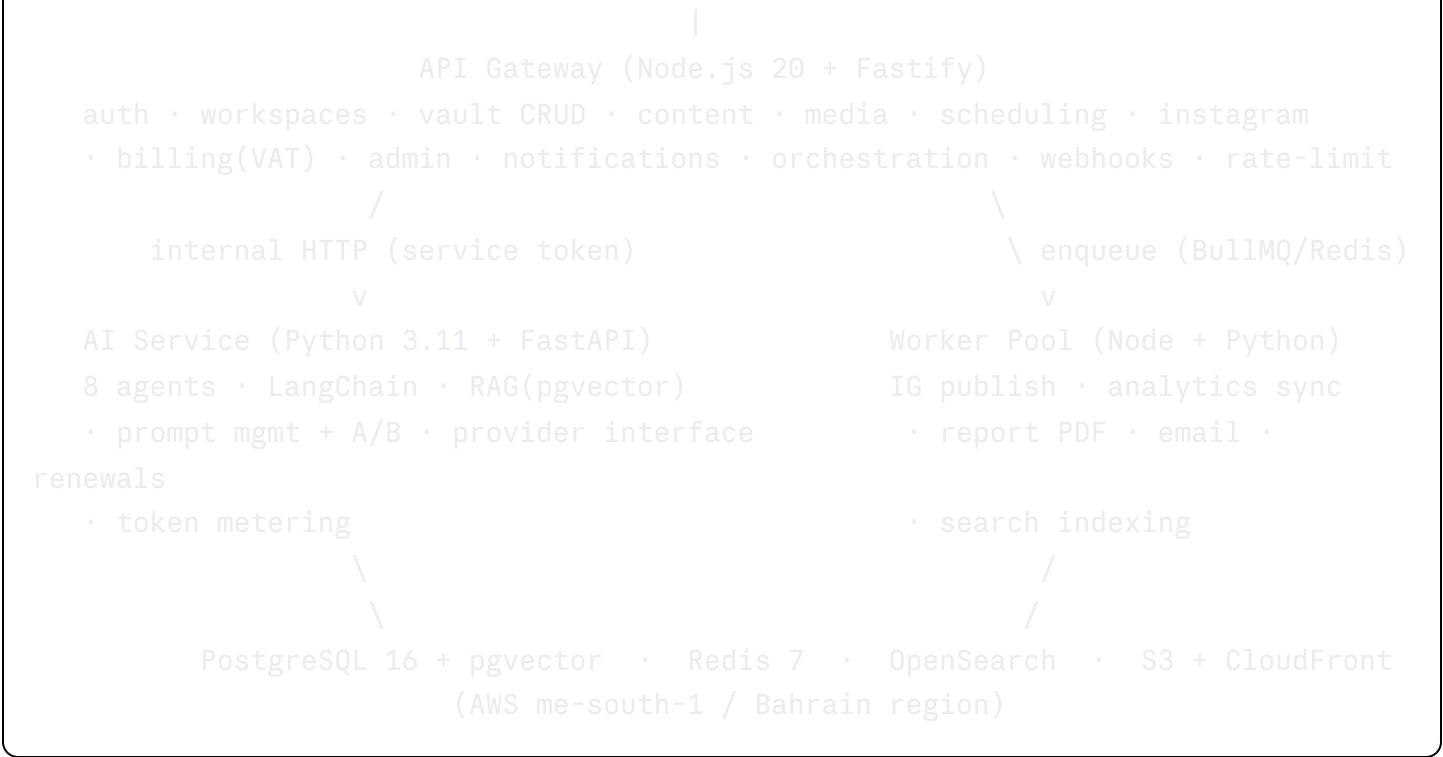
"Unlimited" in marketing = a high fair-use ceiling enforced as a quota (the numbers above). Publishing is additionally bounded by Instagram's per-account 24h ceiling (Section 18).

1.4 The fourteen user types → roles

Businesswomen-style SMB owners, marketers, agencies, and admins. Mapped to roles in Section 8.2.

2. Architecture & Services

Web App (Next.js 14, App Router)
REST + GraphQL over HTTPS



2.1 Service responsibilities (strict seams)

Service	Owns	Must NOT own
Web apps/web	All UI, client state, calling the gateway	Business logic, model calls, secrets
API Gateway apps/api	Auth, data, billing+VAT, media, Instagram, orchestration, webhooks, quota enforcement	Prompt text, model calls
AI Service services/ai	The 8 agents, prompts, RAG, model calls, token accounting	Auth, Stripe, Instagram publishing, persistence (it reads via gateway-provided context or read-only DB)
Workers	Async jobs with retries + DLQ	Synchronous request handling

2.2 Internal contract (gateway → AI service)

Called only by the gateway with a service token (never the browser). All return strict JSON validated by the gateway against the schemas in Section 13.

```

POST /ai/strategy/generate { workspaceId, objectives[], horizonDays }
POST /ai/content/plan      { workspaceId, month, pillars[] }
POST /ai/content/caption  { workspaceId, contentItemId, contentType, brief,
pillar }
POST /ai/content/hashtags { workspaceId, topic, count }
POST /ai/content/reel-script { workspaceId, pillar, topic }
POST /ai/image/prompt     { workspaceId, contentItemId, topic }
POST /ai/analytics/interpret { workspaceId, range, metrics }
POST /ai/recommendations  { workspaceId }
POST /ai/advisor/ask       { workspaceId, question, history[] }
POST /ai/vault/embed       { workspaceId, section, entries[] }
GET  /ai/health · GET  /ai/health/deep

```

Every generative call: retrieve top-K Vault chunks for `workspaceId` → assemble the 8-part system prompt (Section 12.3) → call the model via the provider interface → return structured JSON + a `usage` block `{ promptTokens, completionTokens, model, costMinor }`. The gateway persists the result, writes an `ai_interactions` row, and decrements the workspace's usage counter.

3. Technology Stack (pinned; verify latest stable patch on init, do not change majors)

Concern	Technology
Web	Next.js 14 (App Router) + React 18 + TypeScript (strict)
Styling / UI	Tailwind CSS + Radix UI + design system from the Figma doc (<code>packages/ui-tokens</code>)
Client state	Zustand (UI/session) + TanStack Query v5 (server state); React Hook Form + Zod
Charts / calendar / editor	Recharts · FullCalendar · TipTap
API gateway	Node.js 20 LTS + Fastify; REST (OpenAPI 3.0) + GraphQL; <code>jose</code> (JWT), <code>argon2</code> , <code>otplib</code> (TOTP)
ORM	Prisma 6 (gateway, read+write) · SQLAlchemy (AI service, read-only)
AI service	Python 3.11 + FastAPI + LangChain; <code>pydantic</code> v2
Validation	Zod (TS, shared <code>packages/validation</code>) · Pydantic (Py)

Concern	Technology
Queues	Redis 7 + BullMQ (Node) and a Redis-backed worker (Python)
Database	PostgreSQL 16 + <code>pgvector</code> (HNSW index)
Search	OpenSearch 2 (Arabic analyzer + English)
Storage	AWS S3 (private) + CloudFront (signed URLs)
Payments	CrediMax (MPGS) + BENEFIT/BenefitPay primary (Bahrain); Stripe international fallback
Email	SendGrid (transactional) + AWS SES
Social	Meta Instagram Graph API (current version, Section 18)
Infra	AWS CDK (TypeScript); ECS Fargate, RDS Multi-AZ, ElastiCache, OpenSearch, SQS, Route 53, ACM, region <code>me-south-1</code>
Observability	Sentry (all services) + CloudWatch; Datadog APM optional post-traction
CI/CD	GitHub Actions
Tests	Vitest (TS) · pytest (Py) · Playwright (web e2e)

Hard rules: UUID v7 PKs. Money = integer minor units + ISO-4217 (BHD → ×1000). Embeddings 1536-d, consistent across the Vault. Instagram tokens + PII encrypted at rest (AES-256-GCM). All timestamps UTC ISO-8601.

4. Monorepo Layout (create exactly this)

```
markos/
├─ apps/
│   └─ web/                                # Next.js 14 App Router
│       └─ app/
│           ├── (marketing)/                # public site, pricing
│           ├── (auth)/                     # login, signup, verify, reset
│           ├── [locale]/(app)/             # authenticated product (en|ar)
│           ├── dashboard/ vault/ strategy/ content/ calendar/
│           ├── media/ schedule/ analytics/ ai/ settings/
│           ├── [locale]/(admin)/           # admin portal
│           └─ src/
│               └─ features/                # one folder per area (mirrors routes)
```

components/	# built from Figma component library
lib/	# api client, auth, query client, i18n init
styles/	
next.config.ts	
api/	# Node.js 20 + Fastify gateway
src/	
modules/	# auth, workspaces, vault, strategy,
content,	
	# media, schedule, instagram, analytics,
	# billing, admin, notifications, ai-proxy
workers/	# BullMQ: publish, analytics-sync, report,
email, renewals, index	
common/	# guards, interceptors, error filter, audit,
rate-limit	
tenancy/	# workspace context (AsyncLocalStorage) +
RLS plumbing	
main.ts	
prisma/ (schema.prisma + migrations + seed.ts)	
test/	
services/	
ai/	# Python 3.11 + FastAPI
agents/	# strategist, planner, creator, reel,
image_prompt,	
	# analytics, recommender, advisor (one
module each)	
rag/	# chunking, embeddings, retrieval,
completeness	
prompts/	# versioned templates + A/B + the 8-part
assembler	
providers/	# openai.py, anthropic.py, replicate.py
(behind ModelProvider)	
workers/	# python generation workers
main.py	
packages/	
shared-types/	# TS types + enums shared by web + api
validation/	# Zod schemas (web + api request/DTO
validation)	
api-client/	# typed client the web app uses
ui-tokens/	# design tokens from Figma foundations
config/	# shared tsconfig, eslint, prettier
infra/	# AWS CDK (dev/staging/prod stacks, me-
south-1)	
.github/workflows/	# ci.yml, deploy-staging.yml, deploy-
prod.yml	

```

└─ docker-compose.yml                # postgres+pgvector, redis, opensearch
(local)
└─ Makefile
└─ .env.example
└─ docs/decisions.md                 # log of choices made where this spec was
silent
└─ AGENTS.md                         # short pointer to this file

```

Workspace rules: `apps/*` may import `packages/*`; `packages/*` import nothing upward; the Python AI service shares no TS code but **mirrors the enums and the agent output JSON schemas** exactly.

5. Setup, Commands & Environment

5.1 Bootstrap

```

bash

corepack enable && corepack prepare pnpm@latest --activate
pnpm install
cp .env.example .env                # fill values
make db.up                          # docker compose: postgres+pgvector, redis
pnpm --filter api prisma migrate dev
pnpm --filter api prisma db seed    # plans, a demo workspace, prompt templates
make dev                            # web + api + ai service + workers

```

5.2 Makefile targets

Target	Does
<code>make dev</code>	run web + api + ai + workers together
<code>make verify</code>	typecheck + lint + unit + integration (all services) — the commit gate
<code>make test</code> / <code>make test.e2e</code>	tests / Playwright
<code>make db.up</code> / <code>db.down</code> / <code>db.reset</code>	local infra
<code>make migrate</code> / <code>make seed</code>	Prisma migrate / seed

Target

Does

make build

build web + api + ai images

5.3 Complete .env.example

ini

```
# --- Core ---
NODE_ENV=development
API_PORT=4000
AI_PORT=8000
WEB_BASE_URL=http://localhost:3000
API_BASE_URL=http://localhost:4000
AI_BASE_URL=http://localhost:8000
INTERNAL_SERVICE_TOKEN=change-me          # gateway <-> AI service auth
DEFAULT_LOCALE=ar                         # ar | en
DEFAULT_CURRENCY=BHD

# --- Database ---
DATABASE_URL=postgresql://markos:markos@localhost:5432/markos
DIRECT_DATABASE_URL=postgresql://markos:markos@localhost:5432/markos # migrations
APP_DB_ROLE=markos_app                  # non-superuser, subject to RLS

# --- Redis / Queues ---
REDIS_URL=redis://localhost:6379

# --- Search ---
OPENSEARCH_URL=http://localhost:9200
OPENSEARCH_USER=admin
OPENSEARCH_PASS=admin

# --- Auth ---
JWT_ACCESS_SECRET=change-me
JWT_REFRESH_SECRET=change-me
JWT_ACCESS_TTL=900                      # 15 min
JWT_REFRESH_TTL=2592000                 # 30 days
MFA_ISSUER=MARKOS-AI
GOOGLE_OAUTH_CLIENT_ID=
GOOGLE_OAUTH_CLIENT_SECRET=

# --- AI models (IDs only; never hardcode in code) ---
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
REPLICATE_API_TOKEN=
LLM_PRIMARY_MODEL=gpt-5.4               # workhorse: captions, planning
LLM_FLAGSHIP_MODEL=gpt-5.5              # hardest tasks only
LLM_LONGFORM_MODEL=claude-opus-4-8      # long strategy docs
LLM_CHEAP_MODEL=claude-haiku-4-5        # routing/classification
EMBEDDING_MODEL=text-embedding-3-small  # 1536-d
IMAGE_MODEL_PRIMARY=                    # current OpenAI image model
```

```

IMAGE_MODEL_FALLBACK=                                # SDXL-class on Replicate
AI_PROMPT_CACHE=true
AI_BATCH_FOR_REPORTS=true

# --- Storage / CDN ---
S3_BUCKET=markos-media-dev
S3_REGION=me-south-1
CDN_BASE_URL=https://cdn-dev.markos.bh
SIGNED_URL_TTL=3600

# --- Payments (Bahrain primary) ---
CREDIMAX_MERCHANT_ID=
CREDIMAX_API_PASSWORD=
CREDIMAX_BASE_URL=https://credimax.gateway.mastercard.com
CREDIMAX_API_VERSION=                                # MPGS version
BENEFIT_MERCHANT_ID=
BENEFIT_SECRET=
BENEFIT_BASE_URL=
STRIPE_SECRET_KEY=                                    # international fallback
PAYMENT_WEBHOOK_SECRET=
VAT_RATE_BPS=1000                                     # Bahrain VAT 10% = 1000 basis points
VAT_PRICING_MODE=exclusive                             # exclusive | inclusive

# --- Instagram / Meta ---
META_APP_ID=
META_APP_SECRET=
META_GRAPH_VERSION=v21.0                             # verify current at build
META_REDIRECT_URI=http://localhost:3000/settings/instagram/callback
META_WEBHOOK_VERIFY_TOKEN=

# --- Email ---
EMAIL_PROVIDER=sendgrid
SENDGRID_API_KEY=
SES_REGION=me-south-1
FROM_EMAIL=no-reply@markos.bh

# --- Observability ---
SENTRY_DSN=

```

5.4 Local infra (`(docker-compose.yml)`)

Postgres 16 with `(pgvector)` (init the non-superuser `(markos_app)` role used for RLS), Redis 7, OpenSearch 2. Seed must create the `(markos_app)` role and the RLS policies of Section 9.3.

6. Conventions

6.1 TypeScript / Python

- TS: `strict`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`. No `any`, no non-null `!` outside tests. Shared `tsconfig.base.json` in `packages/config`.
- Python: `mypy --strict` on `services/ai`; Pydantic models for all I/O.

6.2 Naming

- Files `kebab-case.ts`; React components `PascalCase.tsx`; hooks `useThing.ts`.
- DB tables `snake_case` plural; columns `snake_case`; Prisma models `PascalCase` singular.
- API routes `/v1/kebab-case`. Enum values stored as `SCREAMING_SNAKE`.

6.3 API style

- REST under `/v1`, JSON only, UTC ISO-8601. Success: `{ "data": ..., "meta": ... }`. Error: `{ "error": { "code", "message", "details": [] } }` (never leak internals).
- Cursor pagination: `?limit=20&cursor=<opaque>` → `meta.nextCursor`.
- Mutating endpoints that could double-charge accept an `Idempotency-Key` header.

6.4 Errors & logging

- Typed domain exceptions → a global filter maps to the error envelope + HTTP status.
- Structured JSON logs (pino / structlog) with `requestId`, `workspaceId`, `userId`. Never log secrets, tokens, card data, or PII.

6.5 i18n / RTL (Bahrain-first)

- All UI text via `t('namespace:key')` from `packages/i18n`; ship `ar` + `en` for every key in the same PR.
- Default locale **Arabic**; `dir="rtl"` when locale is `ar`; mirror layout, nav, charts, and reading order. Locale-aware dates/numbers; Hijri-aware date display where appropriate.

6.6 Money & VAT

- Store BHD as fils (amount × 1000), `currency` column always set. Compute VAT with `VAT_RATE_BPS`. If `VAT_PRICING_MODE=exclusive`, plan prices are net and VAT is added at checkout and recorded separately; if `inclusive`, derive net = gross × 10000 / (10000 + VAT_RATE_BPS).

6.7 Commits / branches

Conventional Commits (`(feat:)`, `(fix:)`, `(test:)`, `(chore:)`, `(refactor:)`, `(docs:)`). Branch per slice `feat/<area>-<slice>`. PR lists the acceptance criteria it satisfies; CI must pass before merge.

7. FOUNDATION (build first, in order — gate before any feature)

1. **Monorepo + 3 services + packages** per Section 4; shared tsconfig/eslint/prettier; pnpm + Turborepo.
2. **App shells run:** `(web)` renders a localized home (`(ar+en)`); `(api)` `(/v1/health)` OK; `(ai)` `(/ai/health)` OK.
3. **Local infra** via docker-compose; connect `api` → Postgres/Redis/OpenSearch; `ai` → Postgres (read-only) + providers.
4. **Prisma schema + first migration** for ALL tables (Section 10) incl. the `(pgvector)` column + HNSW index on `(knowledge_vault.embedding)`.
5. **Tenancy plumbing** (Section 9): workspace context via AsyncLocalStorage; Prisma middleware sets `(app.current_workspace)`; RLS policies; fail-closed.
6. **Auth** (Section 8): email/password + verification, Google OAuth, JWT (15m) + refresh (30d) rotation w/ reuse detection, argon2id, TOTP MFA for admin/finance roles.
7. **RBAC** (Section 8.2–8.3): roles, permission catalogue, `(@Roles())` + `(@Permissions())` guards.
8. **Shared packages:** `(shared-types)` (enums + core types), `(validation)` (auth + core Zod), `(api-client)` (auth + refresh + workspace header), `(i18n)` (ar+en core), `(ui-tokens)` (Section 7.2).
9. **Provider interface** (Section 13.1) wired to current models; token-accounting skeleton; `(usage_counters)`.
10. **CI/CD** (Section 22 infra): GitHub Actions runs `(make verify)` on PRs; deploy to staging on merge to `(main)`.
11. **Observability:** Sentry in all three services; `(/v1/health)` + `(/v1/health/deep)` (DB/Redis/OpenSearch/AI).
12. **Submit Meta App Review** (parallel, non-code; see Section 18) — start in M0.

7.2 Design tokens (`(packages/ui-tokens)`) — MARKOS brand

ts

```
export const colors = {
  navy: '#1A1A2E', midnavy: '#0F3460', accent: '#E94560',
  bg: '#F8FAFC', card: '#FFFFFF', border: '#D8DEE9',
  text: '#1A1A2E', muted: '#5A6072',
  success: '#22C55E', warning: '#F59E0B', error: '#EF4444',
} as const;
export const spacing = { xs:4, sm:8, md:12, lg:16, xl:24, xxl:32 } as const;
export const radii = { button:6, input:8, card:12, pill:999 } as const;
export const typography = { fontFamily:'Inter', display:'Cal Sans',
  sizes:{ caption:12, body:14, bodyLg:16, h3:18, h2:22, h1:28, display:36 } } as const;
export const elevation = { e1:'0 1px 2px rgba(16,24,40,.06)', e2:'0 4px 12px rgba(16,24,40,.06)' }
```

- Icons: Lucide, 24px default, 1.5 stroke; accent only for active/AI.
- Use an "AI spark" glyph + accent/50 aura wherever AI generates or suggests.

7.3 Foundation acceptance criteria

Register → verify → login (incl. Google) → land on a gated localized shell, on web, in **ar** and **en** (RTL works). Schema deployed with a working HNSW vector index. Isolation test passes (Section 9.4). `make verify` + CI green. Meta App Review submitted.

8. Authentication, Sessions, Roles & Permissions

8.1 Auth flow & endpoints (`modules/auth`)

- Access JWT (15m) claims: `sub` (userId), `workspaceId`, `roles[]`, `iat`, `exp`. Refresh (30d) stored in Redis `refresh:<userId>:<jti>`, rotated on use, reuse-detected (revoke family on reuse).
- MFA: TOTP (otplib), required for admin/finance roles; optional for others.

Method	Path	Purpose
POST	<code>/v1/auth/register</code>	create user + workspace shell + trial; send verification
POST	<code>/v1/auth/verify-email</code>	confirm token
POST	<code>/v1/auth/login</code>	email+password → tokens or MFA challenge
POST	<code>/v1/auth/mfa/verify</code> · <code>/mfa/enroll</code>	complete / begin TOTP
POST	<code>/v1/auth/oauth/google</code>	Google OAuth sign-in/up
POST	<code>/v1/auth/refresh</code> · <code>/logout</code>	rotate / revoke
POST	<code>/v1/auth/password/forgot</code> · <code>/reset</code>	reset flow

8.2 Roles (map the 14 user types)

Roles are per user, scoped to a workspace except platform roles. ★ = MFA required.

Role	Maps to	Scope
<code>OWNER</code>	account owner / SMB owner	workspace
<code>EDITOR</code>	marketer/teammate who creates content	workspace
<code>VIEWER</code>	review-only teammate	workspace
<code>WORKSPACE_ADMIN</code> ★	manages team + billing (Enterprise)	workspace
<code>SUPER_ADMIN</code> ★	MARKOS platform super admin	platform
<code>PRODUCT_ADMIN</code> ★	prompts, plans, content moderation	platform
<code>SUPPORT_ADMIN</code> ★	user/workspace support actions	platform
<code>FINANCE_ADMIN</code> ★	revenue, invoices, refunds	platform
<code>READONLY_ADMIN</code>	internal dashboards read-only	platform

8.3 Permission catalogue (examples; `@Permissions(...)`)

`vault.read|write`, `strategy.generate`, `content.create|publish`, `media.upload|generate_ai`, `schedule.manage`, `analytics.read`, `ai.chat`, `billing.manage`, `team.manage`, plus admin perms `admin.users.*`, `admin.prompts.*`, `admin.plans.*`, `admin.revenue.read`. `WORKSPACE_ADMIN` perms are always workspace-scoped by the tenant context.

9. Multi-Workspace Isolation (correctness — non-negotiable)

9.1 Model

Each account has ≥ 1 **workspace** (Enterprise: up to 10). The workspace is the tenant. Every workspace-owned table carries `workspace_id`.

9.2 Request-time context

Resolve `workspaceId` from the JWT; store in an `AsyncLocalStorage` `WorkspaceContext`; reject authed requests with no resolvable workspace (fail closed). The active workspace can be switched (multi-workspace users) via a header `X-Workspace-Id` validated against the user's memberships.

9.3 Database-enforced RLS

App connects as non-superuser `markos_app`. Per transaction:

```
sql
SET LOCAL app.current_workspace = '<workspaceId>';
```

Every workspace-owned table:

```
sql
ALTER TABLE content_items ENABLE ROW LEVEL SECURITY;
CREATE POLICY ws_isolation ON content_items
    USING (workspace_id = current_setting('app.current_workspace')::uuid);
```

RLS is the backstop; application queries still filter by `workspace_id`.

9.4 Mandatory test

For every workspace-owned table: seed two workspaces; assert workspace A cannot

read/update/delete B's rows through the API. This test must exist before a table's feature is "done".

10. Full Data Model (`schema.prisma`)

UUID v7 PKs; `createdAt`/`updatedAt` everywhere; `deletedAt` for soft-delete; `workspaceId` on workspace-owned tables; money = Int minor units + `currency`.

prisma

```

model User {
  id String @id @default(dbgenerated("uuid_generate_v7()")) @db.Uuid
  email String @unique
  passwordHash String?
  fullName String
  locale Locale @default(AR)
  planId String? @db.Uuid
  planStatus PlanStatus @default(TRIAL)
  trialEndsAt DateTime?
  isVerified Boolean @default(false)
  mfaSecret String? // encrypted
  mfaEnabled Boolean @default(false)
  googleId String? @unique
  lastLoginAt DateTime?
  memberships WorkspaceMember[]
  deletedAt DateTime?
}

model Workspace {
  id String @id @default(...) @db.Uuid
  ownerUserId String @db.Uuid
  name String
  slug String @unique
  instagramAccountId String?
  instagramAccessToken String? // AES-256-GCM encrypted
  instagramTokenExpiresAt DateTime?
  onboardingStatus OnboardingStatus @default(NOT_STARTED)
  onboardingScore Int @default(0)
  vatPricingMode VatMode @default(EXCLUSIVE)
}

model WorkspaceMember { id String @id @default(...) @db.Uuid; workspaceId String @db.Uuid; }

model KnowledgeVault {
  id String @id @default(...) @db.Uuid
  workspaceId String @db.Uuid
  section VaultSection
  key String
  value Json
  embedding Unsupported("vector(1536)")?
  version Int @default(1)
  @@index([workspaceId, section])
}

model Strategy { id String @id @default(...) @db.Uuid; workspaceId String @db.Uuid; }

```

```

model ContentCalendar { id String @id @default(...) @db.Uuid; workspaceId String @db.Uuid;
model Campaign { id String @id @default(...) @db.Uuid; workspaceId String @db.Uuid;

model ContentItem {
  id String @id @default(...) @db.Uuid
  workspaceId String @db.Uuid
  contentType ContentType
  status ContentStatus @default(DRAFT)
  captionEn String? captionAr String?
  hashtags String[]
  callToAction String?
  mediaIds String[] @db.Uuid
  carousel Json? // slides[]
  reelScript Json? // {hook, body, cta}
  contentPillar String?
  campaignId String? @db.Uuid
  aiPromptUsed String?
  scheduledAt DateTime?
  publishedAt DateTime?
  instagramPostId String?
  failureReason String?
}

model MediaAsset {
  id String @id @default(...) @db.Uuid
  workspaceId String @db.Uuid
  type MediaType
  filename String
  s3Key String
  cdnUrl String
  mimeType String
  sizeBytes Int
  width Int? height Int? durationSeconds Int?
}

model InstagramAnalytics {
  id String @id @default(...) @db.Uuid
  workspaceId String @db.Uuid
  contentItemId String? @db.Uuid
  metricType MetricType
  dataDate DateTime
  metrics Json
  syncedAt DateTime
  @@index([workspaceId, metricType, dataDate])
}

model AiInteraction {

```

```

id String @id @default(...) @db.Uuid
workspaceId String @db.Uuid
agent String
promptVersion String
prompt Json
response Json
accepted Boolean? edited Boolean? regenerated Boolean?
tokensIn Int tokensOut Int costMinor Int model String
embedding Unsupported("vector(1536)")?
}

```

```

model Plan { id String @id @default(...) @db.Uuid; name String; priceMinor Int; curr
model Subscription { id String @id @default(...) @db.Uuid; userId String @db.Uuid; g
model Invoice { id String @id @default(...) @db.Uuid; userId String @db.Uuid; gatewa
model Payment { id String @id @default(...) @db.Uuid; workspaceId String @db.Uuid; p
model UsageCounter { id String @id @default(...) @db.Uuid; workspaceId String @db.Uu

model PromptTemplate { id String @id @default(...) @db.Uuid; agent String; version S
model Notification { id String @id @default(...) @db.Uuid; userId String @db.Uuid; c
model AuditLog { id String @id @default(...) @db.Uuid; actorId String? @db.Uuid; act

```

10.1 Enums

```

Locale(EN,AR), PlanStatus(TRIAL,ACTIVE,PAST_DUE,SUSPENDED,CANCELLED)),
OnboardingStatus(NOT_STARTED,IN_PROGRESS,COMPLETE)), VatMode(EXCLUSIVE,INCLUSIVE)),
Role(OWNER,EDITOR,VIEWER,WORKSPACE_ADMIN,SUPER_ADMIN,PRODUCT_ADMIN,SUPPORT_ADMIN,FINANCE_A
DMIN,READONLY_ADMIN)),
VaultSection(COMPANY,STORY,PRODUCTS,AUDIENCE,COMPETITORS,BRAND,TONE,OBJECTIVES)),
ContentType(POST,CAROUSEL,STORY,REEL)),
ContentStatus(DRAFT,IN_REVIEW,APPROVED,SCHEDULED,PUBLISHED,FAILED)),
MediaType(IMAGE,VIDEO,BRAND_ASSET,AI_GENERATED)),
MetricType(ACCOUNT,POST,STORY,REEL,AUDIENCE)), PaymentPurpose(SUBSCRIPTION)),
PaymentStatus(INITIATED,AUTHORIZED,CAPTURED,FAILED,REFUNDED)),
SubStatus(TRIALING,ACTIVE,PAST_DUE,CANCELLED)), InvoiceStatus(DRAFT,PAID,FAILED,VOID)),
UsageMetric(AI_GENERATION,AI_IMAGE,POST_PUBLISH,STRATEGY,STORAGE_BYTES)).

```

11. API Endpoint Catalogue (gateway, **/v1**)

All authed, workspace-scoped, paginated where lists. Validation via Zod. (Auth endpoints in Section 8.1.)

Area	Endpoints
Workspaces	GET/POST /workspaces, PATCH /workspaces/:id, GET /workspaces/:id/usage
Onboarding	GET /onboarding, PUT /onboarding/:module, POST /onboarding/complete
Vault	GET /vault, GET /vault/:section, PUT /vault/:section, GET /vault/score
Strategy	POST /strategies/generate, GET /strategies, GET /strategies/:id, GET /strategies/:id/pdf
Calendar	GET /calendar?month=, POST /calendar/plan, PATCH /calendar/:itemId/move
Content	GET/POST /content, GET/PATCH/DELETE /content/:id, POST /content/:id/generate-caption, /generate-hashtags, /generate-reel-script, POST /content/:id/regenerate/:section, POST /content/:id/approve
Media	GET /media, POST /uploads (presigned), POST /media/generate-ai-image, GET/DELETE /media/:id
Schedule	GET /schedule/queue, POST /content/:id/schedule, PATCH /schedule/:id/reschedule, POST /content/:id/publish-now
Instagram	GET /instagram/status, GET /instagram/connect, GET /instagram/callback, POST /instagram/disconnect
Analytics	GET /analytics/overview?range=, /posts, /posts/:id, /audience, /stories, /reels
AI Consultant	GET /ai/digest, POST /ai/chat, GET /ai/recommendations, POST /ai/report/monthly
Billing	GET /billing/plan, POST /billing/subscribe, POST /billing/change-plan, POST /billing/cancel, GET /billing/invoices, GET /billing/portal
Team	GET /team, POST /team/invite, PATCH /team/:id/role, DELETE /team/:id
Settings	GET/PATCH /settings/account, /settings/workspace, /settings/notifications
Notifications	GET /notifications, POST /notifications/:id/read
Admin	/admin/metrics, /admin/users, /admin/workspaces, /admin/moderation, /admin/ai-usage, /admin/prompts, /admin/plans, /admin/system, /admin/instagram-status, /admin/revenue

Area	Endpoints
Webhooks	POST /webhooks/credimax , /webhooks/benefit , /webhooks/stripe , /webhooks/meta (all signature-verified, idempotent)

12. Knowledge Vault & RAG (build BEFORE the agents)

12.1 Build order

1.

knowledge_vault

 schema + HNSW index on

embedding

.
2. On onboarding save (

PUT /vault/:section

), chunk each answer, call

POST /ai/vault/embed

, store value + 1536-d embedding + version.
3. On every agent call, similarity-search **top 10** chunks relevant to the request context; inject as structured context.
4. Persist agent responses + user feedback/edits into

ai_interactions

 (also embedded) → the continuous-learning corpus.
5. Maintain

onboardingScore

 / completeness; surface gap prompts; refuse generation with a graceful message if the Vault is too sparse to ground the request.

12.2 Retrieval

Cosine similarity over

pgvector

 HNSW; filter by

workspaceId

 (+ optionally

section

); return top 10 with scores; include the most relevant

ai_interactions

 exemplars (high-performing past content) when generating content.

12.3 The 8-part system prompt (assembled for every generative call)

- | | |
|---------------------------------|--|
| 1. AGENT ROLE & CAPABILITIES | (per agent, Section 13) |
| 2. BUSINESS CONTEXT | ← top-10 Vault chunks (company, story, products, audience) |
| 3. BRAND GUIDELINES & TONE | ← Vault brand/tone sections |
| 4. CURRENT MARKETING OBJECTIVES | ← Vault objectives |
| 5. HISTORICAL PERFORMANCE | ← analytics + tagged high/low performers (if relevant) |
| 6. TASK-SPECIFIC INSTRUCTIONS | (the request) |
| 7. OUTPUT FORMAT REQUIREMENTS | ← strict JSON schema (Section 13) the gateway validates |
| 8. QUALITY & GUARDRAILS | ← brand safety, language (ar/en), grounding, length |

Enable prompt caching on parts 1-5 (they repeat per workspace) → ~90% input cost saving.

13. The Eight AI Agents (`services/ai/agents`)

13.1 Provider interface (never hardcode models)

python

```
class ModelProvider(Protocol):
    def generate_text(self, system: str, user: str, *, model: str, temperature: float): ...
    def generate_longform(self, system: str, user: str, *, model: str) -> LLMResult: ...
    def embed(self, texts: list[str], *, model: str) -> list[list[float]]: ...
    def generate_image(self, prompt: str, *, model: str, size: str) -> ImageResult: ...
# Adapters: OpenAIProvider, AnthropicProvider, ReplicateProvider. Model IDs from env
# On provider error -> fall back to the secondary provider (graceful degradation).
# Every result carries usage: {prompt_tokens, completion_tokens, model, cost_minor}.
```

13.2 Per-agent contracts

Each agent declares: model (env key), retrieval scope, and a strict output JSON schema the gateway validates (reject + one auto-retry on invalid JSON).

Content output is **bilingual** (Arabic + English) and tone-locked to the Vault brand/tone. Hashtags tiered niche/mid/broad. Guardrails: language correctness, brand safety, length limits per content type.

13.3 Prompt management, A/B, metering, cost (Section 15 detail)

Versioned `PromptTemplate` rows; A/B variants by `trafficPct`; track accept/edit/regenerate per variant; meter tokens + images per workspace per period → `usage_counters`; cache parts 1-5; route routine copy to `LLM_PRIMARY_MODEL`, reserve flagship/longform for strategy.

14. Onboarding (7 modules → the Vault)

A wizard with progress, skip-and-resume, and a live completeness score. Each module writes to a `VaultSection`. On completion, trigger strategy generation (the "aha" moment).

Module	VaultSection	Key fields
1 Company	COMPANY	name, industry, size, location (Bahrain/GCC), languages
2 Story	STORY	mission, origin, values, USP (textareas w/ counters)
3 Products	PRODUCTS	dynamic rows: name, category, price (BHD), description
4 Audience	AUDIENCE	demographics, interests (tags), pain points
5 Competitors	COMPETITORS	dynamic rows: name, IG handle (verify), notes
6 Brand	BRAND	logo upload, colors, fonts, guidelines PDF, tone words
7 Objectives	OBJECTIVES	checkbox-card goals + KPI targets

Done when: a user completes onboarding; answers persist with embeddings; completeness score + gap prompts work; a test agent call retrieves correct business context; first strategy generates.

15. Prompt Management, A/B & Cost Governance

- Prompts are DB `PromptTemplate` rows (agent, version, body, variantOf, trafficPct, active) — editable + rollback-able in the admin Prompt Manager.
- A/B: assign a variant by `trafficPct`; record `accepted/edited/regenerated` on each `AiInteraction`; expose per-variant rates in admin.
- Metering: every AI call writes tokens + `costMinor` to `ai_interactions` and increments the workspace `UsageMetric` counter. Enforce: soft-warn at 80%, hard-block at 100% with a contextual upgrade prompt.
- Cost levers (implement): prompt caching on parts 1-5; Batch API for monthly reports + bulk planning + re-embedding; model routing (workhorse vs flagship/longform); cache stable retrievals.

16. Feature Areas (endpoints + rules + done-criteria)

Build per the milestone order (Section 22). Each maps to Figma screens (IDs in the Design Document).

16.1 Dashboard & Vault

KPI cards, AI insight widget, upcoming content, quick actions; Vault editor with section tabs,

completeness, version history. **Done:** new-workspace empty state → populated after onboarding; Vault edits re-embed.

16.2 Strategy

`POST /strategies/generate` (Strategist agent, <20s target), strategy hub + detail + PDF export. **Done:** generates a 30/60/90 + pillars grounded in the Vault; exports PDF.

16.3 Content & Calendar

Calendar (month/week/list, drag-drop), AI monthly plan, content creator (caption/hashtags/CTA/carousel/reel), TipTap editor, status workflow draft→in_review→approved, per-section regenerate, comments. **Done:** from a calendar slot, generate a full bilingual, tone-locked item with AI image; move it through the workflow.

16.4 Media Library

Grid + filters + storage meter; presigned `POST /uploads`; AI image generation (Image Prompt agent → image model → S3 → counts to quota); asset detail with usage history. **Done:** upload + AI-generate within plan limits; over-limit shows the limit state.

16.5 Scheduling & Publishing — Section 18.

16.6 Analytics

Sync workers populate `instagram_analytics`; screens: overview (range), posts table, post detail, audience, stories, reels; Analytics Consultant insights + weekly digest + chat Q&A; monthly PDF via worker. **Done:** real metrics render in all views; AI interprets last 30 days; PDF emails.

16.7 AI Consultant

`GET /ai/digest`, `POST /ai/chat` (streaming), `GET /ai/recommendations`. **Done:** digest + chat grounded in Vault + analytics; recommendations are explainable.

16.8 Settings & Team & Admin

Account, workspace, IG integration, billing+VAT, team (Enterprise roles), notifications; admin portal (10 screens per Section 11). **Done:** admin can edit plan limits without deploy and manage/version prompts.

17. Search · Notifications · Jobs · Real-time

- **Search:** OpenSearch indices for members(not v1)/content/media/analytics where useful; primary product search is the content library + media (Postgres) with OpenSearch for larger datasets; Arabic + English analyzers.

- **Notifications:** `NotificationService` with templated bilingual messages over email (SES/SendGrid), in-app, and (post-MVP) push. Triggers: verification, payment receipts (with VAT), renewal reminders, publish success/failure, weekly digest, limit warnings.
 - **Jobs (BullMQ + Redis):** queues `publish`, `analytics-sync`, `report`, `email`, `renewals`, `search-index`, `ai-batch`. Idempotent, retried, DLQ. Workers share the gateway codebase; AI batch jobs run in the Python worker.
 - **Real-time:** AI chat streaming + publish status updates via WebSocket (or SSE) with polling fallback.
-

18. Instagram Integration (highest-risk — build defensively)

18.1 Realities to design around

- **Two-step container publish:** `POST /{ig-user-id}/media` → poll `GET /{container-id}?fields=status_code` until `FINISHED` → `POST /{ig-user-id}/media_publish`.
- **Media must be at a public URL** at publish time → S3 + CloudFront signed URL valid through the publish window.
- **Per-account ceiling** ≈ **25-100 published items / rolling 24h**; Reels + Stories share the bucket; a carousel = 1. Enforce a per-account governor; queue overflow; never silently drop.
- **Reels:** resumable upload on `upload.facebook.com`; 9:16, ~5-90s, H.264/HEVC; `media_type=REELS`.
- **Carousel:** create children `is_carousel_item=true` → parent `media_type=CAROUSEL` with children → publish parent.
- **Stories:** `media_type=STORIES`.
- **Account requirements:** IG Business/Creator linked to a Facebook Page; **Meta App Review** for `instagram_business_content_publish`, `instagram_manage_insights`, `instagram_basic`, `pages_show_list`. **Submit in M0 — 2-4 week lead.**

18.2 What to build

- `InstagramConnectionService`: OAuth (Page → IG Business), encrypted token storage, refresh, revoke, status.
- `PublishingService` worker: a state machine per content type implementing container → poll → publish, idempotent, with the rate governor; statuses map to `ContentStatus`.
- Failure handling (retry 1-2× within Meta's window → fail → alert user → offer reschedule; on repeated container failure create a fresh container). Persist `failureReason`.
- `AnalyticsSyncService` workers pulling account/post/story/reel/audience insights into `instagram_analytics`, respecting rate limits.
- `POST /webhooks/meta` with signature verification + idempotency.

Done when: with Meta approval, a scheduled post AND a scheduled reel publish to a real IG Business account; a forced failure surfaces an alert + reschedule; the daily cap is enforced and visible.

19. Payments, Billing & VAT (Bahrain)

19.1 Gateway abstraction (never call a gateway directly)

```
ts

interface PaymentGateway {
  name: string; // 'CREDIMAX' | 'BENEFIT' | 'STRIPE'
  createCheckout(i: CreateCheckoutInput): Promise<CheckoutSession>; // hosted/token
  capture(ref: string): Promise<PaymentResult>;
  refund(ref: string, amountMinor: number): Promise<PaymentResult>;
  tokenize(i: TokenizeInput): Promise<PaymentToken>; // for auto-renew
  chargeToken(i: ChargeTokenInput): Promise<PaymentResult>;
  verifyWebhook(headers: Record<string,string>, raw: Buffer): WebhookEvent;
}
```

Adapters: `CredimaxAdapter` (MPGS: hosted checkout + tokenization + recurring — primary for Bahrain subscriptions), `BenefitAdapter` (local debit/BenefitPay), `StripeAdapter` (international fallback). New gateway = one adapter; billing code unchanged.

19.2 Subscriptions & lifecycle

4 plans (Section 1.3), monthly + annual (annual = 2 months free), 14-day trial. Recurring via `tokenize` then scheduled `chargeToken` (renewals worker). Lifecycle: upgrade immediate+prorated; downgrade at period end; cancel → service to period end + 90-day data retention; failed payment → 3-day grace → suspend → 30-day hold → delete. Webhooks reconcile status (never trust client success).

19.3 VAT (10%)

Compute VAT with `VAT_RATE_BPS=1000`. `EXCLUSIVE` mode (default): price is net; add VAT at checkout; store `netMinor`/`vatMinor`/`grossMinor` on `Invoice`/`Payment`. `INCLUSIVE`: derive net from gross. Generate compliant VAT invoices (NBR-style: VAT number, net, VAT, gross) and expose under `GET /billing/invoices`. B2B with a valid VAT ID → reverse-charge note.

19.4 Quota enforcement

Every metered action increments `usage_counters` for the workspace period; limits from

`plans.limits` (admin-editable). Soft-warn 80% (+email), hard-block 100% with contextual upgrade. Publishing additionally bounded by Instagram's 24h ceiling.

Done when: a user subscribes (BHD, VAT shown), is auto-renewed, hits a limit → upgrade prompt → prorated upgrade; a compliant VAT invoice is downloadable.

20. Web App Spec (`apps/web`, Next.js 14)

- Routing:** `app/(marketing)`, `app/(auth)`, `app/[locale]/(app)/...`, `app/[locale]/(admin)/...`. Locale `en|ar` in the path; `dir="rtl"` for `ar`.
- State:** TanStack Query for server state (keys include workspaceId + locale); Zustand for UI/session. React Hook Form + Zod for forms (schemas from `packages/validation`).
- Auth storage:** httpOnly refresh cookie + in-memory access token; silent refresh.
- Components:** built from the Figma component library; every screen implements empty/loading (AI skeletons)/error/success/limit states. AI outputs use the AI block (accept/regenerate/edit + spark aura).
- Build per Figma screen IDs:** Auth (AUTH-01-05), Onboarding (OB-01-10), Dashboard/Vault (DASH-01-02), Strategy (STRAT-01-03), Content (CONT-01-06), Media (MEDIA-01-04), Schedule (SCHED-01-04), Analytics (AN-01-06), AI (AI-01-04), Settings (SET-01-06), Admin (ADMIN-01-10).
- Shared:** ~60-70% of non-UI logic (types, api-client, validation, formatting, i18n) is imported, not rewritten.

21. Quality, Testing & Security

21.1 Testing matrix

Layer	Tool	What
Unit	Vitest / pytest	quota math, VAT math, agent output parsing, RAG ranking
Integration	Vitest + test Postgres	every endpoint incl. workspace isolation per table; webhooks; publish state machine (mock Meta)
Contract	Zod / Pydantic	request/response + agent output schemas

Layer	Tool	What
Web e2e	Playwright	onboarding→strategy, content creation, schedule→publish (sandbox), subscribe→limit→upgrade — both locales
Load	k6	toward NFR targets before M6
Security	pen test	before M1 launch and before M5 billing
Localization	manual + snapshot	every screen in ar (RTL) + en

Gates: PRs blocked unless make verify green. Money + publishing paths require integration tests before merge. 80%+ coverage on api domain logic and the AI agents' parsing/guardrails.

21.2 Security & compliance checklist (must all hold at launch)

- ☐ RBAC on every endpoint; admin/finance roles require MFA.
- ☐ Workspace isolation proven by automated tests for every workspace-owned table.
- ☐ argon2id passwords; JWT 15m + refresh 30d rotation w/ reuse detection.
- ☐ Instagram tokens + PII AES-256-GCM at rest; KMS; secrets in Secrets Manager.
- ☐ Private S3 + signed CloudFront (1h); TLS 1.3; HSTS/CSP/X-Frame-Options.
- ☐ OWASP Top-10 (parameterized queries, output encoding, SameSite + CSRF).
- ☐ Idempotency on charge endpoints; webhook signature verification + idempotency (Credimax/Benefit/Stripe/Meta).
- ☐ Per-user/IP/endpoint rate limits.
- ☐ Immutable audit log for admin + financial actions.
- ☐ **Bahrain PDPL:** data-subject export + erasure (30-day), consent flow, privacy policy; data hosted in me-south-1 where feasible.
- ☐ **VAT:** correct 10% handling + compliant invoices.
- ☐ Pen test passed before each major launch.

22. Build Order (milestones — execute in order; each gate must pass)

#	Milestone	Delivers	Gate (Done when)
M0	Foundation (§7)	Monorepo, 3 services, infra, auth, schema+pgvector, tenancy+RLS, shared packages, design tokens, CI/CD, Meta review submitted	§7.3 acceptance
M1	Onboarding + Vault + RAG (§12,14)	7-module wizard, Vault storage+embeddings+versioning+score, brand upload, RAG proven, provider interface live	A grounded test agent call returns correct business context; completeness + gaps work
M2	AI Content Engine (§13,16.2-16.4)	All 8 agents, strategy gen, calendar, content creation (bilingual), TipTap workflow, AI image pipeline, prompt A/B + token metering	From a calendar slot → full tone-locked bilingual item + AI image, moved through workflow; strategy PDF exports
M3	Instagram (§18)	OAuth connect, publish state machine (image/carousel/story/reel), queue UI, rate governor, failure alerts	Real post + reel publish to a test IG Business account; forced failure → alert + reschedule
M4	Analytics (§16.6)	sync workers, AN screens, Analytics Consultant + digest + chat, monthly PDF, learning loop	Real metrics render; AI interprets 30 days; PDF emails; perf data feeds the Vault
M5	Billing + VAT + Admin (§19,16.8)	Credimax/Benefit/Stripe adapters, subscriptions, VAT invoices, metering+enforcement, admin portal (10 screens), admin RBAC	Subscribe (BHD+VAT) → hit limit → prorated upgrade; admin edits plan limits w/o deploy + manages prompts
M6	Beta + Launch	private beta, perf to NFR, security audit (OWASP), PDPL + VAT verified, load test, Arabic/RTL QA	NFR met; audit passed; Starter + Growth live in Bahrain

23. Reference Map — PRD → build

PRD element	Where built
8 AI agents (§8.2)	<code>services/ai/agents</code> (§13)
Knowledge Vault RAG (§8.3)	<code>knowledge_vault</code> + <code>services/ai/rag</code> (§12)
Continuous learning (§8.4)	<code>ai_interactions</code> + few-shot from tagged performers (M4)
7-module onboarding (§4.2)	§14; Figma OB-01-10
Instagram publishing (§4.8)	§18; M3
Analytics (§4.9)	§16.6; M4; Figma AN-01-06
Plans & billing (§12)	§19; M5 — Bahrain CrediMax/BENEFIT + VAT
Admin portal (§13)	§16.8; M5; Figma ADMIN-01-10
Security (§14)	§21.2 — + Bahrain PDPL
30-week roadmap (§15)	§22 milestones
Design system & screens (§11)	Figma Design Document; <code>packages/ui-tokens</code> + <code>apps/web</code>
Bahrain market	local pricing (§1.3), CrediMax/BENEFIT (§19), VAT 10% (§19.3), me-south-1 (§3,§21), Arabic/RTL (§6.5)

*End of specification. Start at **MO** and submit Meta App Review the same week. Build the Knowledge Vault + one grounded agent call (M1) before agent breadth — it de-risks everything. Keep `make verify` green; honor the milestone gates; where this document is silent, choose the smallest conventional option and log it in `/docs/decisions.md`.*